

Reviewer Pack — Permutation Polynomial Degree Bounds ABP Size for NC¹ Functi...

Entry f56efd7da9d8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 10:34:42 UTC

Permutation Polynomial Degree Bounds ABP Size for NC¹ Functions

Entry ID: f56efd7da9d8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 10:34:42 UTC

1. Conjecture

Field A (mathematical branch): Permutation Polynomials over Finite Fields **Field B** (complexity object): Algebraic Branching Programs (ABPs)

Statement:

For every Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ in NC¹, the minimal degree of a permutation polynomial over GF(2ⁿ) representing f is at most the size of the smallest ABP computing f . Conversely, if an ABP has size s , then the minimal degree of such a polynomial is $\Omega(s / \log n)$.

Rationale (proposer's reasoning):

Permutation polynomials over finite fields capture symmetry in Boolean functions, while ABP size reflects algebraic complexity. Linking their growth rates could reveal structural constraints on NC¹ computations via algebraic representations.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8cfead6d46488a3f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.randint(0, 1) for _ in range(2**n)]

    def abp_size(f, n):
        if n == 1:
            return 1
        size = float('inf')
        for i in range(n):
            for j in range(i+1, n):
                f_i_j = [f[i] ^ f[j] for i, j in zip(range(2**n), range(2**n))]
                size = min(size, abp_size(f_i_j[:2**(i+j)], i+j) + 1)
        return size

    def permutation_polynomial_degree(f, n):
        x = [random.randint(0, 2**n - 1) for _ in range(n)]
        y = [f[i] for i in range(2**n)]
        A = []
        for j in range(n):
            row = []
            for i in range(2**n):
                row.append(x[j] ^ x[(i + (1 << j)) % 2**n])
            A.append(row)
        B = y[:]
        for _ in range(n):
            pivot_col = max(range(n), key=lambda col: abs(sum(A[row][col] for row in range(2**n))))
            if A[0][pivot_col] == 0:
                return float('inf')
            for i in range(1, 2**n):
                factor = A[i][pivot_col] / A[0][pivot_col]
                for j in range(n):
                    A[i][j] -= factor * A[0][j]
                B[i] -= factor * B[0]
            A.pop(0)
```

```

        B.pop(0)
    return n

n = 40
f = generate_boolean_function(n)
abp_s = abp_size(f, n)
poly_d = permutation_polynomial_degree(f, n)

if poly_d > abp_s or poly_d < abp_s / math.log(n):
    return {
        "metric_name": "degree",
        "metric_value": poly_d,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"ABP size: {abp_s}, Poly degree: {poly_d}"
    }

return {
    "metric_name": "degree",
    "metric_value": poly_d,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"degree mismatch\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} < 0.8")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing validation of conjecture. | next: Increase timeout duration and re-run test with additional debugging instrumentation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111213
2	propose	ollama_remote	qwen3:8b	0	0	97502
3	preregistration	ollama_remote	qwen3:8b	0	0	24287
4	novelty	ollama_remote	qwen3:8b	0	0	20765
5	novelty	ollama_remote	qwen3:8b	0	0	12367
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10325
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9785
8	judge	ollama_remote	qwen3:8b	0	0	31370

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 317614 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/f56efd7da9d8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f56efd7da9d8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f56efd7da9d8.tar.gz (if generated)